GitHub Actions 자동 수집 가이드

PC와 Docker를 켜 두지 않아도 매일 수집이 돌고, 결과 덤프가 Release에 자동으로 올라가게 하는 방법(Release + Actions)을 정리했습니다. Release 덤프와 Actions의 차이, 비용, 장단점, 도입 순서까지 담았습니다.

날짜	2026-10-06
상태	**제안 (아직 적용 전)** — 팀 결정 후 workflow 파일을 추가합니다
기준 코드	main `81d748e` (수집: `save_listings.collect_all`, 스케줄러: `scheduler.py`)
관련 문서	`261002_7_덤프파일_가이드` · `261006_8_수집스케줄러_백업_변경사항` · `261006_9_DB덤프_Release_사용법`

목차

1 **한눈에 보기**
지금 방식과 무엇이 달라지는지, 세 줄 요약

2 **Release 덤프와 GitHub Actions** 개념
무엇이고, 둘이 어떻게 다른지, 같이 쓰면 어떻게 되는지

3 **자동 수집은 정확히 어떻게 도나** 동작
매일 06:00의 7단계, 수집 10단계별로 무엇을 얼마나 받는지

4 **비용과 한도** 무료
Actions 월 2,000분, Release 용량, 클라우드 DB가 안 맞는 이유

5 **장단점과 비교**
지금(수동) · Release + Actions · 클라우드 DB + Actions

6 **API 키는 어떻게 나눠 쓰나**
공용 키 1개(수집) vs 각자 키(개발·화면 기능)

7 **도입 순서와 역할** 따라 하기
저장소 소유자·팀원이 할 일, Claude 없이 운영하는 법

8 **주의할 점 · 자주 묻는 질문**
해외 IP 차단, 실행 시각, 로컬 스케줄러 끄기 등

1. 한눈에 보기

Release

데이터를 담아 나르는 **상자**. DB를 파일(덤프)로 떠서 GitHub에 보관·공유

Actions

정해진 시각에 명령을 대신 실행하는 **로봇**. GitHub 서버에서 돌아감

같이 쓰면

로봇이 **매일 상자를 채워** 줌. 사람·PC 없이 수집 → 덤프 → 업로드

> **요약:** 지금은 지윤님 PC가 오전 6시에 켜져 있어야(또는 켜진 뒤 보충) 수집되고, 덤프도 사람이 직접 올립니다. Actions를 붙이면 **GitHub 서버가 매일 수집하고 새 덤프를 Release에 올립니다.** 팀원은 지금처럼 Release에서 받아 복원하기만 하면 됩니다.

지금 vs Release + Actions

	지금 (수동)	Release + Actions
수집하는 곳	지윤님 PC의 backend 컨테이너	GitHub 서버 (임시 컴퓨터)
필요 조건	PC 켜짐 + Docker 실행 중	**없음** — PC가 꺼져 있어도 됨
덤프 업로드	사람이 직접, 필요할 때	**자동, 매일**
팀원이 할 일	Release에서 받기 → 복원	같음 (명령 하나짜리 스크립트로 단축 가능)
API 키	지윤님 `.env`	공용 키 1개를 GitHub Secrets에
비용	0원	0원 (4장)

> **오해하기 쉬운 점** — PC 없이 되는 건 **수집**입니다. 내 화면에서 최신 데이터를 보려면 지금처럼 **Docker(DB) + 덤프 복원 + backend·frontend 실행**이 필요합니다.

2. Release 덤프와 GitHub Actions

둘은 같은 종류의 선택지가 아닙니다. **Release 덤프는 데이터를 보관·전달하는 방법**이고, **Actions는 작업을 대신 실행하는 도구**입니다. 그래서 "둘 중 하나"가 아니라 **같이 쓰는 관계**입니다.

2-1. Release 덤프 = DB를 파일로 떠서 GitHub 자료 게시판에 올린 것

- **덤프**: DB 전체(테이블 구조 + 데이터)를 파일 하나로 저장한 것. `pg_dump`로 만들고 `pg_restore`로 되살립니다. 1.65GB DB가 약 66MB 파일이 됩니다.
- **Release**: 저장소의 "자료 게시판". 코드와 별개로 큰 파일(파일당 2GB 미만)을 첨부합니다. 자세한 내용은 `261006_9_DB덤프_Release_사용법`.

2-2. GitHub Actions = GitHub 서버가 대신 실행해 주는 자동화

- 저장소에 `.github/workflows/*.yml` 파일을 넣으면, GitHub이 조건에 맞춰 **임시 컴퓨터(Ubuntu)**를 만들어 그 안의 명령을 실행합니다.
- 조건: **예약**(매일 06:00), **push**(코드 올릴 때), **수동 버튼**(Run workflow) 등.
- 작업이 끝나면 임시 컴퓨터는 사라집니다 → **결과는 반드시 어딘가에 따로 저장**해야 합니다. 여기서는 Release.

2-3. 차이

	Release 덤프	GitHub Actions
정체	저장·공유 방법 (파일)	자동 실행 도구 (로봇)
하는 일	DB를 파일로 보관하고 팀원에게 전달	정해진 시각에 명령(수집 등)을 실행
혼자서 수집?	**불가** — 누군가 수집해서 올려야 함	**가능** — 단, 결과를 둘 곳이 필요
혼자서 보관?	**가능**	**불가** — 실행 후 컴퓨터가 사라짐
사람 손	올리기·받기 모두 수동	설정만 해 두면 자동

2-4. 같이 쓰면

```
[매일 06:00 KST · GitHub 서버]
 Actions 시작 → 임시 Postgres → 어제 Release 덤프 복원 → collect_all 수집
            → pg_dump → 새 Release 자동 업로드 (예: db-dump-20261007)
                                          |
 [팀원 PC]  최신 Release 다운로드 → pg_restore ◀┘   ← 이 부분만 사람이
```

> Actions가 "지윤님 PC에서 수집하고 덤프 올리던 일"을 대신 맡고, Release는 지금처럼 결과물 보관함 역할을 그대로 합니다.

3. 자동 수집은 정확히 어떻게 도나

3-1. 매일 06:00에 일어나는 7단계

#	단계	내용
①	임시 컴퓨터 준비	Ubuntu + Python 설치, `requirements.txt` 설치 (캐시하면 1분 이내)
②	빈 Postgres 띄우기	로컬과 같은 `postgres:16`을 서비스 컨테이너로
③	어제 덤프 복원	최신 Release의 `.dump`를 받아 `pg_restore` (약 40초)
④	`collect_all` 실행	아래 3-2의 10단계. 실행 기록은 `collect_runs`에 `trigger="GitHub Actions"` 등으로 남김
⑤	새 덤프 만들기	`pg_dump -Fc` (약 15초)
⑥	Release 업로드	새 태그 `db-dump-YYYYMMDD`로 업로드. 설명에 행 수·실패 단계 기록. 오래된 Release는 최근 7개만 남기고 삭제
⑦	정리	임시 컴퓨터 삭제 (자동)

> **③이 중요한 이유** — 빈 DB에서 시작하면 LH 2010년~ 상세정보를 처음부터 받아야 해서 호출 한도 때문에 며칠이 걸립니다. 어제 덤프를 이어받으면 무거운 부분은 **새로 생기거나 바뀐 것만** 받습니다.

3-2. collect_all 10단계 — 무엇을, 얼마나

#	무엇을	어디로	매일 받는 범위
1	청약홈 공고 × 주택형	`listings` (applyhome)	**전체 기간 매번 다시** → 있으면 갱신, 없으면 추가
2	경쟁률·당첨가점·특별공급	`historical_results`	**전체 기간 매번 다시** (같은 방식)
3	LH 공고 (2010년~)	`listings` (lh), `lh_notice_details`	목록은 전부, **상세·공급정보는 새 공고·바뀐 공고만**. 한도에 걸리면 멈추고 다음 날 이어서
4	SH 분양·임대 게시판	`sh_notices`	목록 훑고 **본문·첨부는 새 글만**
5	SH 모집 접수기간	`listings` (sh)	최근 90일 모집 공고
6	파인 대출 상품 + 금리	`loan_products`, `loan_rate_options`	전체 다시 (양이 적음)
7	청약통장·신청·당첨자 통계	`bankbook_stats`, `subscription_stats`	**마지막 저장 달부터만** (그 달은 다시)
8	가공 뷰 재계산	`listing_features`, `cutoff_stats` 등	1~7 결과로 다시 계산
9	공고문 PDF → 계약금	`notice_documents`, `notice_costs` 등	**접수 예정·중이면서 아직 안 읽은 공고만**
10	화면용 계약금 칸	`listings.deposit_*`	9의 결과를 옮김

> **안전장치 (이미 코드에 있음)** — ① 한 단계가 실패해도 나머지는 계속 진행하고 실패 단계는 `collect_runs.failed_steps`에 남김 ② "있으면 갱신, 없으면 추가"(upsert)라 두 번 돌려도 중복 없음 ③ LH 상세 호출이 실패하면 예전 원본을 덮어쓰지 않고 다음 수집 때 재시도

4. 비용과 한도

팀 저장소는 **비공개(Private)**입니다. 비공개 저장소는 Actions 실행 시간에 월 한도가 있습니다.

항목	무료 범위	우리 사용량	결과
Actions 실행 시간	비공개 저장소 **월 2,000분**	1회 약 15~30분 × 30일 = **약 450~900분**	**무료**
Release 파일	파일당 2GB 미만, 전체 용량 제한 없음	66MB × 최근 7개 ≈ 460MB	**무료**
Release 다운로드	제한 없음	팀원 다운로드	**무료**
Secrets	무료	API 키 2~3개	**무료**

> **1회 시간 추정 근거** — 10/06 보충 수집(나흘 치)이 **11:52~12:02, 약 10분** 걸렸습니다(`261006_8` 문서). 여기에 준비·복원·덤프·업로드 약 3~5분을 더하고, GitHub 서버가 미국에 있어 한국 API 응답이 느릴 수 있는 점을 감안해 15~30분으로 잡았습니다. **첫 수동 실행에서 실제 시간을 꼭 확인**하세요.

> **한도를 넘으면?** 개인 무료 계정은 결제 한도가 기본 **$0**이라 요금이 나가지 않고, 그 달 남은 기간 Actions가 멈출 뿐입니다. 모자라면 **격일·평일만 실행**하거나, 무거운 단계(청약홈 전체 기간)를 "최근 N개월만"으로 바꿉니다 (`save_applyhome_listings` 에 `from_date` 옵션이 이미 있음).

4-1. 클라우드 DB는 왜 안 맞나

	Release + Actions	클라우드 DB + Actions
필요한 용량	덤프 66MB (파일)	**DB 그대로 약 1.65GB**
무료 한도	파일당 2GB 미만	Supabase·Neon 등 약 **0.5GB** → **초과**
비용	**0원**	유료 플랜 필요 (보통 월 $19~25부터)

클라우드 무료 플랜 수치는 서비스마다 자주 바뀌니 실제 도입 시 각 사이트에서 다시 확인하세요. 용량을 줄이려면 `raw` JSON·공고문 텍스트를 빼야 하는데, 팀의 "원본 그대로 저장" 방향과 맞지 않습니다.

5. 장단점과 비교

5-1. 장점

장점	설명
PC 없이 수집	누구의 PC·Docker도 켜져 있을 필요 없음. "6시에 꺼져 있어서 나흘 치가 빠지는" (10/03~10/06) 일이 사라짐
0원	Actions·Release 모두 무료 범위 안
날짜별 백업 자동	매일 덤프가 남아서 수집 버그로 데이터가 망가져도 **어제 덤프로 되돌릴 수 있음**
팀원끼리 영향 없음	각자 로컬 DB라 한 명이 DB를 망가뜨려도 다른 사람은 그대로
바뀌는 게 적음	팀원 작업 방식(Release에서 받아 복원)이 지금과 같음
API 호출 한 곳	공용 키 1개로 하루 한 번 → LH 하루 10,000건 한도를 나눠 쓰지 않음

5-2. 단점

단점	설명	대응
복원은 수동	수집은 자동이지만 내 로컬 DB 반영은 직접. 안 하면 며칠 전 데이터	"최신 덤프 받아 복원" 스크립트
로컬 데이터 덮어씀	복원하면 테스트로 만든 `user_profiles` 등이 사라짐	복원 전 백업, 또는 스크립트에서 사용자 테이블 보존
Docker는 여전히 필요	화면을 보려면 로컬 Postgres 필요	—
해외 IP 차단 위험	GitHub 서버는 미국. SH 홈페이지·LH가 막으면 그 단계만 실패	첫 수동 실행으로 확인. 막히면 그 단계만 로컬 수집
실행 시각 지연	예약 실행이 몇 분~수십 분 늦을 수 있음	하루 단위 데이터라 영향 적음
관리 포인트	Secrets는 저장소 소유자만 등록·교체 가능. 실패 메일 확인 필요	소유자 1명이 담당

5-3. 세 방식 비교

	지금 (수동)	Release + Actions	클라우드 DB + Actions
PC 꺼져도 수집	✗	✓	✓
비용	0원	**0원**	**유료 (1.65GB)**
최신 데이터 반영	수동 복원	수동 복원	**자동**
로컬 Docker 필요	✓	✓	✗
날짜별 백업	가끔	**매일 자동**	따로 설정
팀원끼리 영향	없음	없음	**공용 DB라 있음**

> **추천:** 지금 팀 상황(각자 로컬 개발, 비용 0원)에는 **Release + Actions**. 실제 서비스로 배포할 때는 어차피 서버용 DB가 필요하니 그때 클라우드 DB를 다시 검토합니다.

6. API 키는 어떻게 나눠 쓰나

수집을 Actions 한 곳으로 모으면 **팀원은 수집 API를 직접 부르지 않습니다**(덤프만 받음). 그래서 키를 용도별로 나눕니다.

키	누구 것	어디에	이유
`APPLYHOME_API_KEY`, `FINE_API_KEY` (자동 수집)	**공용 1개**	GitHub Secrets	하루 한 번, 한 곳에서만 호출. 각자 전체 수집을 돌리면 같은 데이터를 여러 번 받는 낭비 + DB가 서로 달라짐
같은 키 (개발·테스트)	**각자**	각자 `backend/.env`	수집 코드를 고치며 여러 번 호출할 때, 공용 키 한도를 지키기 위해
`KAKAO_REST_API_KEY`	**각자**	각자 `backend/.env`	출퇴근 조회 1번 = 주소→좌표 2번. 화면을 누를 때마다 호출
`ODSAY_API_KEY`	**각자**	각자 `backend/.env`	무료 하루 한도가 작고, 서버 키는 **PC의 공인 IP를 등록**해야 해서 공유 자체가 어려움
`OPENAI_API_KEY`	**각자**	각자 `backend/.env`	유료라 공유하면 비용 책임이 애매함

> **한도가 모자랄 때** 여러 사람의 키를 돌려 쓰는 건 이용약관 위반이 될 수 있습니다. data.go.kr에서 **운영계정(트래픽 증가)**을 신청하세요.

> **키는 채팅·노션에 붙여넣지 마세요.** `.env`는 `.gitignore`에 있어 커밋되지 않습니다(지금까지 커밋 기록 없음 확인). 공용 키는 저장소 소유자가 Secrets에만 넣습니다.

6-1. 카카오·ODsay 키 발급 (출퇴근 기능용)

키	발급 순서
카카오 REST	developers.kakao.com 로그인 → 내 애플리케이션 → 애플리케이션 추가 → **앱 키**의 **REST API 키** 복사 → **제품 설정 → 카카오맵 → 사용 설정 ON** (안 켜면 403)
ODsay	lab.odsay.com 가입 → 애플리케이션 등록 → 플랫폼 **서버**, 내 PC 공인 IP 입력(ifconfig.me에서 확인) → 키 복사

두 키를 `backend/.env`에 넣은 뒤 **backend를 재시작**해야 반영됩니다. 카카오 키만 있으면 주소→좌표까지, 대중교통 경로·소요시간은 ODsay 키가 있어야 나옵니다.

7. 도입 순서와 역할

7-1. 처음 한 번

#	누가	할 일	방법
1	개발 (누구나)	workflow 파일 작성	`.github/workflows/daily-collect.yml` — 처음엔 **수동 버튼(`workflow_dispatch`)만** 켜 둠
2	개발 (누구나)	로컬 스케줄러 끄는 설정	`.env` 에 `SCHEDULER_ENABLED=false` 같은 값으로 backend의 06:00 수집을 끔 (중복 방지)
3	개발 (누구나)	팀원용 복원 스크립트	"최신 Release 받기 → 복원 → 행 수 확인"을 명령 하나로
4	**저장소 소유자**	Secrets 등록	저장소 → Settings → Secrets and variables → Actions → `APPLYHOME_API_KEY` , `FINE_API_KEY` (LH 다른 계정이면 `LH_API_KEY`)
5	**저장소 소유자**	첫 실행 (수동)	Actions 탭 → workflow 선택 → **Run workflow**. 로그에서 단계별 `[저장] … N행` , `[실패]` , 전체 시간 확인
6	팀	매일 실행 켜기	해외 IP 차단 없고 시간이 한도 안이면 `schedule: cron: "0 21 * * *"` 추가 (UTC 21:00 = KST 06:00)

7-2. workflow 뼈대 (예시 · 적용 전)

```
name: daily-collect
on:
  workflow_dispatch:            # 수동 버튼 (처음엔 이것만)
  # schedule:
  #   - cron: "0 21 * * *"      # 매일 06:00 KST (확인 후 켜기)
jobs:
  collect:
    runs-on: ubuntu-latest
    timeout-minutes: 120
    services:
      db:
        image: postgres:16
        env: { POSTGRES_USER: cheongyak, POSTGRES_PASSWORD: cheongyak, POSTGRES_DB: cheongyak }
        ports: ["5432:5432"]
    env:
      DATABASE_URL: postgresql+psycopg2://cheongyak:cheongyak@localhost:5432/cheongyak
      APPLYHOME_API_KEY: ${{ secrets.APPLYHOME_API_KEY }}
      FINE_API_KEY: ${{ secrets.FINE_API_KEY }}
      GH_TOKEN: ${{ github.token }}
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12", cache: pip }
      - run: pip install -r backend/requirements.txt
      - name: 어제 덤프 복원
        run: |
          gh release download --pattern "*.dump" -O latest.dump   # 최신 Release
          pg_restore -h localhost -U cheongyak -d cheongyak --no-owner latest.dump
      - name: 수집
```

```
        working-directory: backend
        run: python -m app.data_pipeline.save_listings
      - name: 새 덤프 업로드
        run: |
          pg_dump -h localhost -U cheongyak -d cheongyak -Fc -f cheongyak_$(date +%Y%m%d).dump
          gh release create db-dump-$(date +%Y%m%d) cheongyak_*.dump --title "DB 덤프 $(date +%F) (자동)"
```

뼈대만 보인 것입니다. 실제 파일에는 PGPASSWORD 설정, 같은 날 재실행 시 태그 중복 처리, 오래된 Release 정리, 실패 단계 요약을 Release 설명에 적는 부분이 더 들어갑니다. Release를 만들려면 workflow 권한에 `contents: write`가 필요합니다.

7-3. 평소 · Claude 없이 운영

상황	할 일
매일 수집	**아무것도 안 함** — GitHub이 자동 실행. Claude·VS Code·내 PC 모두 필요 없음
최신 데이터가 필요할 때	팀원이 복원 스크립트 실행 (또는 `261006_9` 문서 4장 명령)
실패 메일이 옴	Actions 탭 → 실패한 실행 → 로그. 대부분 원인이 그대로 나옴 (키 만료, 사이트 차단 등)
API 키 만료	재발급 후 소유자가 Secrets 교체 (data.go.kr 키는 보통 활용기간 1~2년)
수집 코드 수정	코드만 push하면 다음 실행부터 새 코드 사용. workflow는 그대로

> Claude는 workflow·스크립트를 **만들 때**만 필요하고, **돌아갈 때**는 필요 없습니다. 만든 뒤에는 순수한 GitHub 기능이라 팀원 누구나 관리할 수 있습니다.

8. 주의할 점 · 자주 묻는 질문

주의할 점	내용
해외 IP 차단	GitHub 서버는 미국에 있습니다. SH 홈페이지·LH API·공고문 PDF 다운로드가 해외 접속을 막으면 그 단계만 실패합니다(나머지는 계속). 첫 수동 실행에서 확인하세요.
cron은 UTC	KST 06:00 = UTC 21:00 전날 → `"0 21 * * *"`. 예약 실행은 몇 분~수십 분 늦을 수 있습니다.
로컬 스케줄러 끄기	Actions로 옮긴 뒤에도 backend의 06:00 스케줄러가 켜져 있으면 각자 PC에서 중복 수집됩니다(키 한도 소모).
작업 시간 한도	작업 하나는 최대 6시간. 빈 DB에서 시작하면 LH 전체 수집이 며칠 걸리므로 **반드시 덤프 이어받기**로 시작합니다.
60일 비활성	공개 저장소는 60일간 커밋이 없으면 예약 실행이 멈춥니다(비공개는 해당 없음, 프로젝트 종료 후 이야기).
개인정보	덤프에 `user_profiles`(이메일·자산)가 들어갈 수 있습니다. 저장소를 공개로 바꾸면 Release도 공개되므로, 그 전에 덤프에서 사용자 테이블을 빼야 합니다.

Q. PC가 꺼져 있어도 정말 수집되나요?

네. 수집은 GitHub 서버에서 돌아갑니다. 다만 그 결과를 내 화면에서 보려면 Docker를 켜고 최신 덤프를 복원해야 합니다.

Q. 매일 처음부터 다 받나요?

반만 그렇습니다. 청약홈 공고·경쟁률(1·2단계)은 매일 전체 기간을 다시 받고, LH 상세·SH 본문·통계·공고문 PDF처럼 무거운 부분은 새로 생기거나 바뀐 것만 받습니다(3-2).

Q. 수집이 일부 실패하면 덤프가 안 올라가나요?

제안: **일부 단계 실패는 업로드하고 Release 설명에 실패 단계를 적습니다.** 덤프 복원이나 `pg_dump` 자체가 실패하면 업로드하지 않습니다. 어느 경우든 GitHub이 실패 메일을 보냅니다.

Q. 지금 내 로컬은 바로 쓸 수 있나요?

main `81d748e`을 받았다면 DB를 `cheongyak_20261006_2.dump`로 복원해야 합니다. 예전 구조 DB(테이블 4개)에 새 코드를 띄우면 `column … does not exist` 오류가 나고, 1분 뒤 보충 수집도 실패합니다.

2026-10-06 · 팀 저장소 JiYoon241111/cheongyak-platform (Private) · 수치 근거: 덤프파일 가이드(DB 1.65GB → 덤프 약 63~66MB), 수집 스케줄러 변경사항(보충 수집 약 10분), Release db-dump-20261006-2(66MB). Actions·클라우드 무료 한도는 도입 시점에 다시 확인하세요.